```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report
import pickle
import os

# ------------------ LOAD DATA ------------------
data = pd.read_csv("supply_chain_data.csv")

# clean column names
data.columns = data.columns.str.strip().str.replace(" ", "_")

# remove duplicate columns if any
data = data.loc[:, ~data.columns.duplicated()]

# drop unnecessary columns
data = data.drop(columns=["SKU", "Product_Type", "Supplier_name", "Routes"], errors='ignore')

# handle missing values
data = data.dropna()

# ------------------ ENCODING ------------------
label_encoders = {}
for col in data.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le

# ------------------ TARGET ------------------
data["Defect_Risk"] = (data["Defect_rates"] > data["Defect_rates"].mean()).astype(int)

# features & target
X = data.drop(columns=["Defect_Risk", "Defect_rates"])
y = data["Defect_Risk"]

# ------------------ SPLIT ------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------ SCALING ------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ------------------ TRAIN MODEL ------------------
model = LogisticRegression(max_iter=5000)
model.fit(X_train_scaled, y_train)

# ------------------ PREDICTION (TEST DATA) ------------------
y_pred = model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# ------------------ SAVE MODEL ------------------
os.makedirs("models", exist_ok=True)

with open("models/model.pkl", "wb") as f:
    pickle.dump(model, f)

with open("models/encoders.pkl", "wb") as f:
    pickle.dump(label_encoders, f)

with open("models/scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)

print("\nModel trained and saved successfully!")
```

```
Accuracy: 0.4

Classification Report:
               precision    recall  f1-score   support
```

```
           0       0.45      0.45      0.45        11
           1       0.33      0.33      0.33         9

    accuracy                           0.40        20
   macro avg       0.39      0.39      0.39        20
weighted avg       0.40      0.40      0.40        20


Model trained and saved successfully!
```

```
# ------------------ NEW DATA PREDICTION ------------------

def predict_defect_risk(new_data_dict):
    # load saved objects
    model = pickle.load(open("models/model.pkl", "rb"))
    scaler = pickle.load(open("models/scaler.pkl", "rb"))
    encoders = pickle.load(open("models/encoders.pkl", "rb"))

    # Get the feature names from the global X dataframe (used during training)
    # This assumes X is available in the global scope as a pandas DataFrame
    global X
    expected_columns = X.columns

    # convert to dataframe
    new_df = pd.DataFrame([new_data_dict])

    # Reindex the new_df to match the columns of the training data
    # Fill any missing columns with 0. For real-world, more sophisticated imputation might be needed.
    new_df = new_df.reindex(columns=expected_columns, fill_value=0)

    # encode categorical columns
    for col in new_df.columns:
        if col in encoders:
            # Ensure the column is of object type before attempting to encode
            if new_df[col].dtype == 'object':
                # If a category in new_df[col] is not in encoders[col].classes_,
                # transform will raise a ValueError. For simplicity, we assume valid inputs.
                new_df[col] = encoders[col].transform(new_df[col])

    # scale
    new_scaled = scaler.transform(new_df)

    # predict
    prediction = model.predict(new_scaled)[0]

    return "High Risk" if prediction == 1 else "Low Risk"


# -------- SAMPLE INPUT --------
# Modified sample_input to include all features seen during training
# based on the kernel state's X dataframe (20 columns) and error message.
sample_input = {
    "Product_type": 1,      # Changed to integer as 'Product_type' in X is already numerical
    "Price": 500,
    "Availability": 20,
    "Number_of_products_sold": 100,
    "Revenue_generated": 50000,
    "Customer_demographics": 1,   # Changed from 'Rural' to integer as 'Customer_demographics' in X is already numerical
    "Stock_levels": 30,
    "Lead_time": 5,                   # Renamed from "Lead_times" to "Lead_time" as per error message
    "Order_quantities": 200,
    "Shipping_times": 3,
    "Shipping_costs": 50,
    "Location": "Mumbai",
    "Inspection_results": "Pass",
    "Transportation_modes": "Road",
    "Costs": 1000,                      # Added missing feature (numerical)
    "Manufacturing_costs": 200,         # Added missing feature (numerical)
    "Production_volumes": 500,          # Added missing feature (numerical)
    "Manufacturing_lead_time": 10,      # Added missing feature (numerical)
    "Logistics_costs": 75,              # Added missing feature (numerical)
    "Product_innovation_score": 0.8     # Added missing feature (numerical)
}

print("\nPrediction for new data:", predict_defect_risk(sample_input))
```

```
Prediction for new data: Low Risk
```